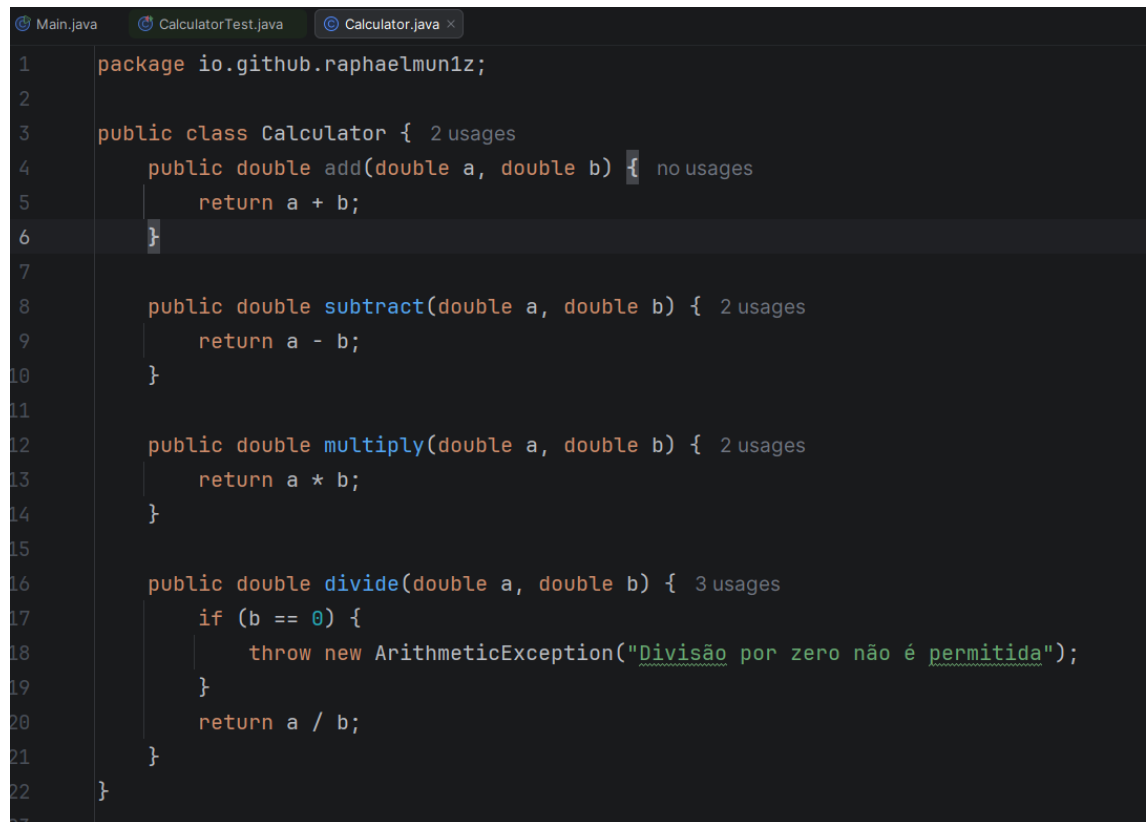


JUNIT ATIVIDADE: ESTENDENDO A CALCULADORA

1. Estenda as funcionalidades da calculadora incluindo operações de subtração, multiplicação e divisão.



```
1 package io.github.raphaelmuniz;
2
3 public class Calculator { 2 usages
4     public double add(double a, double b) { no usages
5         return a + b;
6     }
7
8     public double subtract(double a, double b) { 2 usages
9         return a - b;
10    }
11
12    public double multiply(double a, double b) { 2 usages
13        return a * b;
14    }
15
16    public double divide(double a, double b) { 3 usages
17        if (b == 0) {
18            throw new ArithmeticException("Divisão por zero não é permitida");
19        }
20        return a / b;
21    }
22 }
```

2. Crie novos cenários de teste para cobrir as funcionalidades criadas.
3. Lembre-se de incluir casos de teste tanto para situações válidas, quanto para situações inválidas.

```
@Test
public void testSubtract() {
    assertEquals(expected: 5, calculator.subtract(a: 10, b: 5), message: "10 - 5 deve ser 5");
    assertEquals(expected: -2, calculator.subtract(a: 3, b: 5), message: "3 - 5 deve ser -2");
}

@Test
public void testMultiply() {
    assertEquals(expected: 50, calculator.multiply(a: 10, b: 5), message: "10 * 5 deve ser 50");
    assertEquals(expected: 0, calculator.multiply(a: 10, b: 0), message: "Qualquer número multiplicado por 0 é 0");
}

@Test
public void testDivide() {
    assertEquals(expected: 2, calculator.divide(a: 10, b: 5), message: "10 / 5 deve ser 2");
    assertEquals(expected: 2.5, calculator.divide(a: 5, b: 2), message: "5 / 2 deve ser 2.5");
}

@Test
public void testDivideByZero() {
    assertThrows(ArithmeticException.class, () -> {
        calculator.divide(a: 10, b: 0);
    }, message: "Deve lançar ArithmeticException ao dividir por zero");
}
```